

Apostila Completa de CSS

Guia visual com dezenas de exemplos práticos

Propriedades • Seletores • Box Model • Flexbox • Grid
Transform • Transitions • Animations • Filters • Efeitos modernos

Iniciante

Intermediário

Avançado

1. Introdução ao CSS

CSS (Cascading Style Sheets — Folhas de Estilo em Cascata) é a linguagem que descreve a **apresentação visual** dos documentos HTML. Enquanto o HTML define a *estrutura* (parágrafos, listas, imagens), o CSS define *como tudo aparece*: cores, fontes, tamanhos, espaçamentos, layouts, animações, efeitos 3D.

Por que "Cascading"? Porque vários estilos podem se aplicar ao mesmo elemento e o navegador resolve qual prevalece em "cascata", seguindo regras de **especificidade** e **ordem**.

1.1 Três formas de aplicar CSS

```
/* 1) Externo — RECOMENDADO. Reutilizável e cacheável */
<link rel="stylesheet" href="estilo.css">

/* 2) Interno — bom para protótipos */
<style>
  body { background: #f5f5f5; }
</style>

/* 3) Inline — evite. Difícil de manter */
<p style="color: red; font-size: 18px;">Texto</p>
```

1.2 Sintaxe básica

```
seletor {
  propriedade: valor;  /* uma declaração */
  propriedade: valor;
}

/* Exemplo real */
h1 {
  color: #1e40af;
  font-size: 32px;
  text-align: center;
}
```

1.3 Comentários

```
/* Comentário de uma linha */
```

```
/* Comentário  
de várias  
linhas */
```

2. Seletores

Seletores escolhem **quais elementos** receberão os estilos. Dominar seletores é fundamental para escrever CSS limpo.

2.1 Seletores básicos

Seletor	Exemplo	O que seleciona
Universal	*	Todos os elementos
Tipo / tag	p	Todos os <p>
Classe	.btn	Elementos com class="btn"
ID	#header	Único elemento com id="header"
Atributo	input[type="email"]	Inputs do tipo email
Lista	h1, h2, h3	Vários seletores (mesma regra)

2.2 Combinadores

Combinador	Exemplo	Descrição
Descendente (espaço)	article p	Qualquer <p> dentro de <article>
Filho direto (>)	ul > li	filho direto de
Irmão adjacente (+)	h1 + p	Próximo irmão imediato
Irmãos gerais (~)	h1 ~ p	Todos os <p> irmãos seguintes

2.3 Pseudo-classes (estados)

```
a:hover      /* mouse em cima */
a:active     /* sendo clicado */
input:focus  /* recebendo foco */
input:checked /* checkbox/radio marcado */
input:disabled /* desabilitado */

li:first-child /* primeiro filho */
li:last-child  /* último filho */
li:nth-child(2) /* segundo filho */
li:nth-child(odd) /* ímpares */
li:nth-child(even) /* pares */
li:nth-child(3n+1) /* fórmula: 1, 4, 7... */

p:not(.destaque) /* negação */
:is(h1, h2, h3) /* qualquer um deles */
div.has(> img) /* div que contém img */
```

2.4 Pseudo-elementos (partes do elemento)



Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Praesent eget odio nec eros
tincidunt.

```
p::before {
  content: "★ "; /* requer content */
  color: gold;
}

p::first-letter {
  font-size: 22pt;
  font-weight: bold;
  color: #dc2626;
}

/* Outros: ::first-line, ::placeholder,
   ::selection, ::marker (bullet de lista)
*/
```

2.5 Especificidade

Quando duas regras se aplicam ao mesmo elemento, ganha a de **maior especificidade**.
Calculada como (a, b, c, d):

- **a** = inline style (`style="..."`) → 1000
- **b** = IDs → 100 cada
- **c** = classes, atributos, pseudo-classes → 10 cada
- **d** = tipo (tag), pseudo-elementos → 1 cada

!important sobreescreve qualquer especificidade — mas evite. Sinal de que o CSS está mal

estruturado.

3. Cores e Unidades

3.1 Formatos de cor

```
color: red;           /* nome (140+ nomes) */
color: #ff0000;       /* hexadecimal */
color: #f00;          /* hex curto = #ff0000 */
color: #ff0000aa;     /* hex com alpha */
color: rgb(255, 0, 0); /* RGB */
color: rgba(255, 0, 0, 0.5); /* + opacidade 0..1 */
color: hsl(0, 100%, 50%); /* matiz, saturação, luminosidade */
color: hsla(0, 100%, 50%, .5);
color: oklch(0.7 0.2 30); /* moderno, perceptualmente uniforme */
color: currentColor; /* herda a cor do texto */
color: transparent;
```

Dica HSL: para clarear/escurecer uma cor, só mude o terceiro valor (luminosidade). Para variações da mesma cor, mantenha matiz e saturação iguais.

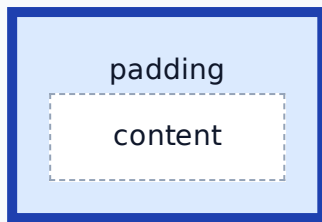
3.2 Unidades de medida

Unidade	Tipo	Descrição	Quando usar
px	Absoluta	Pixels da tela	Bordas, sombras finas
%	Relativa	% do elemento pai	Larguras fluidas
em	Relativa	× font-size do pai	Padding/margin proporcional
rem	Relativa	× font-size do <code><html></code>	Tipografia (acessibilidade)
vw / vh	Viewport	1% largura/altura da janela	Hero, fullscreen
vmin / vmax	Viewport	Menor/maior dimensão	Layouts responsivos
ch	Relativa	Largura do "0"	Largura de texto
fr	Grid	Fração do espaço livre	CSS Grid

4. Box Model

Todo elemento HTML é uma caixa retangular composta por quatro camadas concêntricas:

content (conteúdo) → **padding** (espaço interno) → **border** (borda) → **margin** (espaço externo)



```
.box {  
  width: 160px;  
  padding: 16px;      /* interno: empurra  
                      conteúdo */  
  border: 5px solid #1e40af;  
  margin: 20px;      /* externo: empurra  
                      vizinhos */  
  background: #dbeafe;  
}
```

4.1 box-sizing – CRÍTICO

Por padrão, `width` conta apenas o **content**. Padding e border são **somados** à largura final — o que confunde muito. Por isso quase todo projeto usa:

```
*, *::before, *::after { box-sizing: border-box; }  
  
/* Agora width: 200px inclui padding e border */
```

4.2 padding e margin – 1 a 4 valores

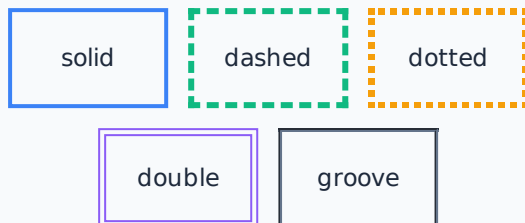
```
padding: 10px;          /* todos os lados */  
padding: 10px 20px;     /* vertical | horizontal */  
padding: 10px 20px 30px; /* topo | lados | base */  
padding: 10px 20px 30px 40px; /* topo direita base esquerda (relógio) */  
  
/* Individuais */  
padding-top: 10px;  
padding-right: 20px;  
padding-bottom: 30px;  
padding-left: 40px;  
  
/* margin: auto centraliza horizontalmente blocos com largura */  
margin: 0 auto;
```

4.3 border-radius – cantos arredondados



```
border-radius: 4px;           /* cantos suaves */
border-radius: 50%;           /* círculo (em quadrado) */
border-radius: 50% 10% 50% 10%; /* TL TR BR BL */
border-radius: 30px 0 30px 0; /* diagonais */
border-radius: 50% 50% 0 0;   /* arco superior */
```

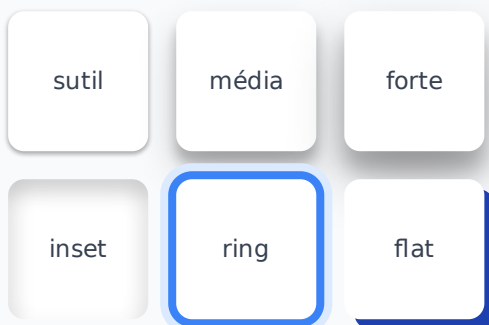
4.4 border – estilos



```
border: 2px solid #3b82f6;
border: 3px dashed #10b981;
border: 3px dotted #f59e0b;
border: 4px double #8b5cf6;

/* Por lado */
border-top: 1px solid #ccc;
border-right: 2px dashed #aaa;
```

4.5 box-shadow – sombras



```
/* offsetX offsetY blur spread cor */
box-shadow: 0 1px 3px rgba(0,0,0,.12);
box-shadow: 0 10px 20px rgba(0,0,0,.19);

/* inset = sombra interna */
box-shadow: inset 0 4px 12px rgba(0,0,0,.3);

/* Múltiplas sombras (vírgula) */
box-shadow: 0 1px 3px rgba(0,0,0,.12),
            0 1px 2px rgba(0,0,0,.24);

/* "Ring" usando spread sem blur */
box-shadow: 0 0 0 4px #3b82f6;
```

5. Tipografia

TÍTULO FORTE

Texto em itálico

Sublinhado ondulado

Com sombra

Gradient text

```
p {
  font-family: "Helvetica", Arial, sans-serif;
  font-size: 16px;          /* tamanho */
  font-weight: 700;         /* 100-900 ou bold */
  font-style: italic;
  line-height: 1.5;         /* altura da linha
(sem unidade) */
  letter-spacing: 2px;      /* entre letras */
  word-spacing: 4px;        /* entre palavras */
  text-align: center;       /* left | right |
justify */
  text-transform: uppercase;
  text-decoration: underline wavy red;
  text-indent: 20px;
  text-shadow: 2px 2px 4px rgba(0,0,0,.3);
  white-space: nowrap;      /* não quebra linha
*/
  word-break: break-word;
}

/* Texto com gradiente */
.title {
  background: linear-gradient(90deg, #3b82f6,
#ec4899);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
```

5.1 Importar fontes (Google Fonts)

```
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap"
rel="stylesheet">
```

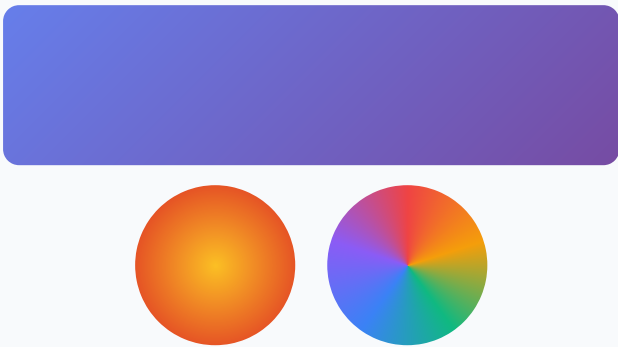
```
body { font-family: "Inter", sans-serif; }
```

6. Backgrounds e Gradientes

6.1 Propriedades de background

```
.hero {  
  background-color: #1e40af;  
  background-image: url("foto.jpg");  
  background-repeat: no-repeat; /* repeat | repeat-x | repeat-y */  
  background-position: center center; /* top, bottom, x% y%, 50% 50% */  
  background-size: cover; /* cobre | contain | 100% auto */  
  background-attachment: fixed; /* parallax */  
  background-origin: padding-box;  
  background-clip: border-box;  
  
  /* SHORTHAND */  
  background: #1e40af url("foto.jpg") center/cover no-repeat fixed;  
}
```

6.2 Gradientes



```
/* Linear: direção, cor1, cor2, ... */  
background: linear-gradient(135deg, #667eea,  
#764ba2);  
background: linear-gradient(to right, red, blue);  
background: linear-gradient(90deg, red 0%, yellow  
50%, green 100%);
```

```
/* Radial: do centro para fora */  
background: radial-gradient(circle, #fbbf24,  
#dc2626);  
background: radial-gradient(ellipse at top, #fff,  
#000);
```

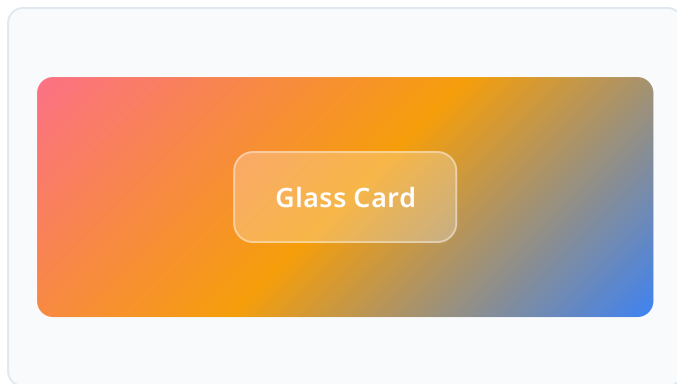
```
/* Conic: gira em torno do centro */  
background: conic-gradient(from 0deg,  
red, orange, yellow, green, blue, purple, red);
```



```
/* Múltiplas cores */  
background: linear-gradient(90deg,  
#ff9a9e, #fad0c4, #fbc2eb, #a18cd1);
```

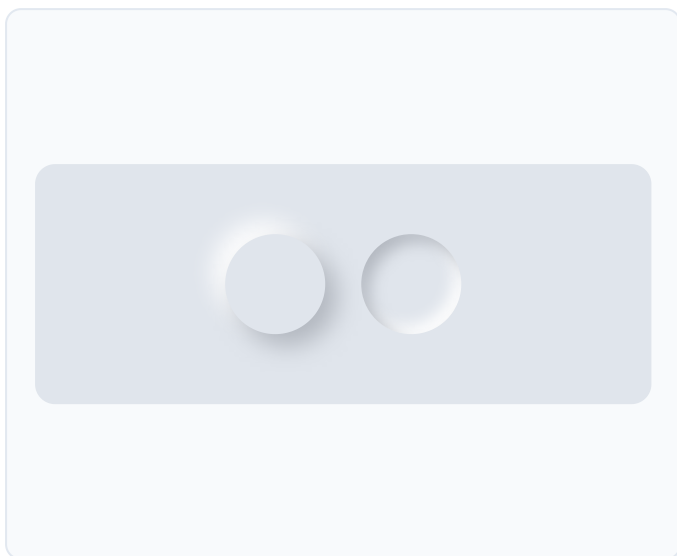
```
/* Repeating gradient (listras) */  
background: repeating-linear-gradient(45deg,  
#1e40af 0, #1e40af 10px,  
#3b82f6 10px, #3b82f6 20px);
```

6.3 Glassmorphism – efeito vidro



```
.glass {  
  background: rgba(255,255,255,.25);  
  backdrop-filter: blur(8px);    /* borra o que  
  está atrás */  
  -webkit-backdrop-filter: blur(8px);  
  border: 1px solid rgba(255,255,255,.4);  
  border-radius: 10px;  
}
```

6.4 Neumorfismo (soft UI)



```
.bg { background: #e0e5ec; }  
  
/* Saliente */  
.btn-out {  
  box-shadow: 6px 6px 12px #b3b8c0,  
             -6px -6px 12px #ffffff;  
}  
  
/* Afundado */  
.btn-in {  
  box-shadow: inset 4px 4px 8px #b3b8c0,  
             inset -4px -4px 8px #ffffff;  
}
```

7. Display e Position

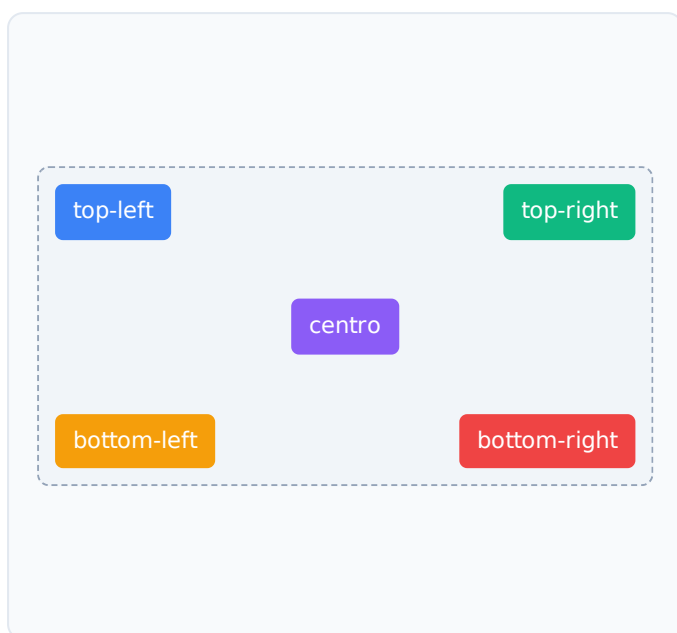
7.1 display – fluxo do elemento

Valor	Comportamento	Exemplos típicos
block	Ocupa a linha inteira, aceita width/height	div, p, h1, section
inline	Fica na linha, ignora width/height	span, a, strong
inline-block	Inline mas aceita width/height	botões customizados
flex	Container flexbox (1D)	navbar, cards alinhados
grid	Container grid (2D)	layouts complexos
none	Some do layout (não ocupa espaço)	esconder
contents	Remove a caixa, mantém filhos	quebrar wrapper

7.2 position – posicionamento

Valor	Descrição
static (padrão)	Segue o fluxo normal. top/left são ignorados.
relative	Move em relação à posição original. Não afeta vizinhos.
absolute	Sai do fluxo. Posiciona em relação ao ancestral posicionado mais próximo.
fixed	Fixo na viewport (não rola com a página).
sticky	Híbrido: relative até atingir um limite, então fixed.

Exemplo: position absolute



```
.parent { position: relative; } /* "ancora" o filho */

.child {
  position: absolute;
  top: 0; left: 0; /* canto */
  bottom: 0; right: 0; /* preenche tudo */
}

/* Centralizar com transform */
.center {
  position: absolute;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}
```

Exemplo: sticky header

```
.menu {
  position: sticky;
  top: 0; /* gruda no topo ao rolar */
  z-index: 100; /* ordem de empilhamento */
}
```

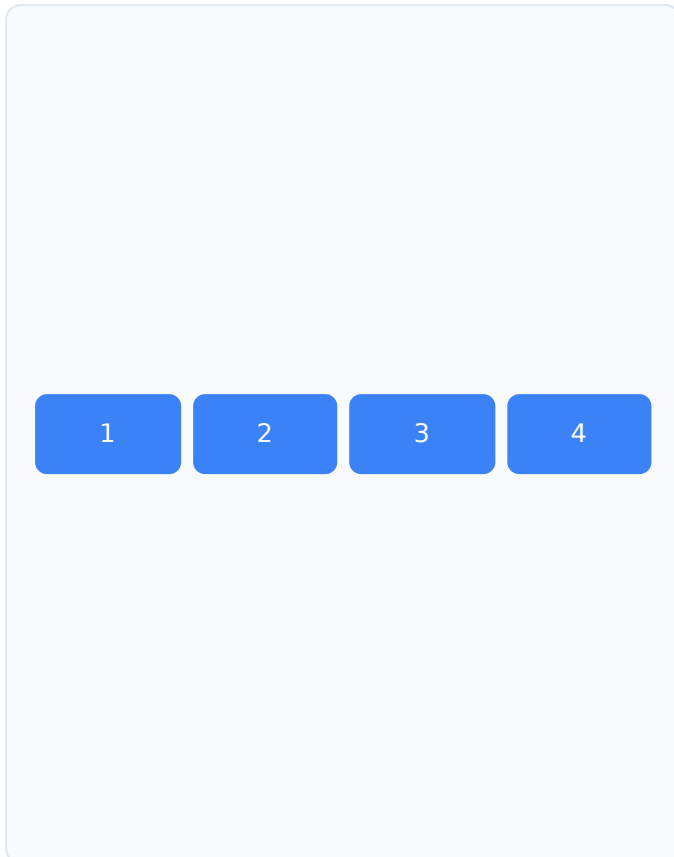
7.3 z-index

Controla a ordem de empilhamento de elementos posicionados (não-static). Maior fica por cima. Funciona apenas em elementos com `position` diferente de `static`.

8. Flexbox (layout 1D)

Sistema **unidimensional** para alinhar e distribuir itens em uma linha ou coluna. Resolve 90% dos problemas de alinhamento.

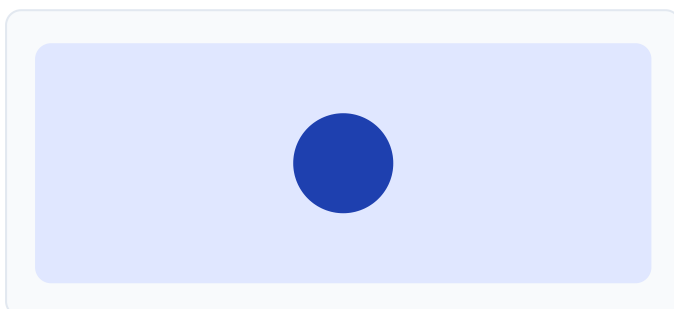
8.1 Propriedades do container



```
.container {
  display: flex;
  flex-direction: row; /* row | row-reverse |
column | column-reverse */
  flex-wrap: wrap; /* nowrap | wrap |
wrap-reverse */
  gap: 8px; /* espaço entre itens */

  justify-content: center; /* eixo principal:
flex-start | center
| flex-end
| space-between
| space-around
| space-evenly */
  align-items: center; /* eixo cruzado:
stretch (padrão) |
flex-start
| center | flex-end | baseline */
  align-content: space-between; /* só com várias
linhas (wrap) */
}
```

8.2 Centralização perfeita – o famoso "santo graal"



```
.center {
  display: flex;
  justify-content: center; /* horizontal */
  align-items: center; /* vertical */
  height: 100vh;
}
```

8.3 Propriedades dos itens

```
.item {
  order: 2;          /* muda a ordem visual */
  flex-grow: 1;      /* cresce para ocupar sobra */
  flex-shrink: 0;     /* 0 = não encolhe */
  flex-basis: 200px; /* tamanho base antes de crescer/encolher */

  /* SHORTHAND: grow shrink basis */
  flex: 1 0 200px;
  flex: 1;           /* ocupar igualmente */
  flex: auto;         /* baseado no conteúdo */

  align-self: flex-end; /* sobrescreve align-items só deste item */
}
```

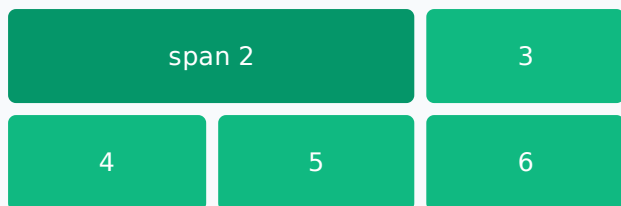
8.4 Padrão: navbar com logo + menu + ações

```
.navbar {
  display: flex;
  align-items: center;
  gap: 20px;
}
.navbar .logo { flex: 0 0 auto; } /* tamanho fixo */
.navbar .menu { flex: 1; }        /* ocupa o meio */
.navbar .actions { flex: 0 0 auto; }
```

9. CSS Grid (layout 2D)

Sistema **bidimensional** — linhas E colunas ao mesmo tempo. Ideal para layouts complexos de página inteira.

9.1 Grid básico

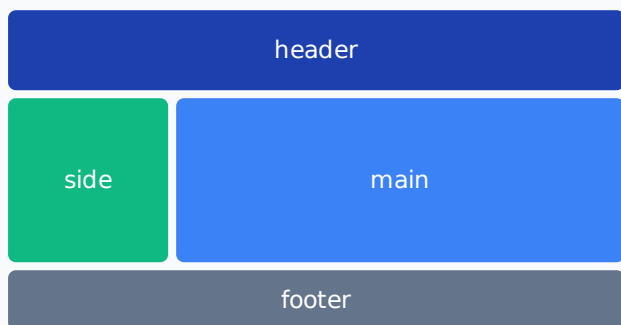


```
.grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr); /* 3  
  colunas iguais */  
  grid-template-rows: 100px auto;  
  gap: 6px; /* row-gap  
  column-gap */  
}  
  
.item:nth-child(1) {  
  grid-column: span 2; /* ocupa 2 colunas */  
}
```

9.2 Sintaxes de grid-template-columns

```
grid-template-columns: 100px 200px 100px; /* pixels fixos */  
grid-template-columns: 1fr 2fr 1fr; /* frações */  
grid-template-columns: repeat(4, 1fr); /* 4 iguais */  
grid-template-columns: repeat(3, minmax(200px, 1fr));  
  
/* RESPONSIVO mágico (cards que se ajustam sozinhos) */  
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
/* ^ cabe quantos couber, mínimo 250px */
```

9.3 Grid template areas – layout nomeado



```
.page {  
  display: grid;  
  grid-template-columns: 80px 1fr;  
  grid-template-rows: 40px 1fr 30px;  
  grid-template-areas:  
    "header header"  
    "side main "  
    "footer footer";  
}  
  
.header { grid-area: header; }  
.side { grid-area: side; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

9.4 Posicionamento de itens

```
.item {
  grid-column: 1 / 3;      /* da linha 1 até 3 */
  grid-column: 1 / span 2; /* da linha 1, ocupa 2 */
  grid-row: 2 / 4;
  grid-area: 2 / 1 / 4 / 3; /* row-start col-start row-end col-end */

  justify-self: center; /* alinha individual horizontal */
  align-self: end;      /* alinha individual vertical */
}

/* No container */
.grid {
  justify-items: center; /* itens no eixo horizontal */
  align-items: stretch; /* itens no eixo vertical */
  place-items: center; /* atalho dos dois */
}
```

10. Transform – o coração das animações

A propriedade `transform` aplica **transformações geométricas** (2D ou 3D) ao elemento. **Não afeta o layout** dos outros elementos (ao contrário de mudar width, top, etc.). É **acelerada por GPU** — ideal para animações fluidas.

Por que usar transform em vez de top/left? top/left disparam recálculo de layout (lento). `transform` e `opacity` rodam na GPU em uma camada separada. Sempre prefira transform para animar.

10.1 Funções 2D – tudo em ação



Função	Sintaxe	O que faz
translate(x, y)	<code>translate(20px, 50px)</code>	Move horizontal e vertical
translateX(x)	<code>translateX(50%)</code>	Move só no eixo X
translateY(y)	<code>translateY(-10px)</code>	Move só no eixo Y
scale(x, y)	<code>scale(1.5)</code> ou <code>scale(2, 0.5)</code>	Aumenta/reduz
scaleX / scaleY	<code>scaleX(-1)</code>	Escala um eixo (negativo = espelha)
rotate(ang)	<code>rotate(45deg)</code>	Gira (deg, rad, turn)
skew(x, y)	<code>skew(20deg, 10deg)</code>	Inclina/distorce
skewX / skewY	<code>skewX(15deg)</code>	Inclina em um eixo
matrix(a,b,c,d,e,f)	<code>matrix(1,0,0,1,0,0)</code>	Combina todas em uma só

10.2 Combinando transformações

Você pode encadear várias funções em uma única declaração. **A ordem importa!**

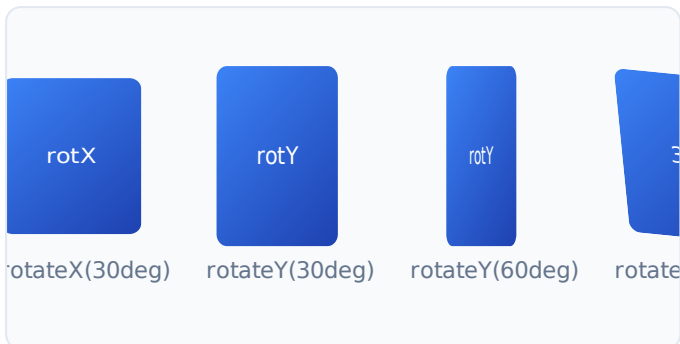
```

.elem {
  transform: rotate(15deg) scale(1.2) translateX(10px);
  /* aplicada da DIREITA para a ESQUERDA:
    1º translateX, 2º scale, 3º rotate */
}

/* DIFERENTE de: */
transform: translateX(10px) scale(1.2) rotate(15deg);
/* aqui rotate é aplicado primeiro */

```

10.3 Funções 3D



```

/* O PAI define a perspectiva */
.cena { perspective: 700px; }

/* O FILHO faz a rotação */
.card {
  transform: rotateY(30deg);
}

```

Função 3D	O que faz
translate3d(x, y, z)	Move em 3 eixos (z = profundidade). Otimizada pela GPU.
translateZ(z)	Aproxima/afasta do observador
scale3d(x, y, z)	Escala em 3 eixos
rotateX(ang)	Gira em torno do eixo X (de pé)
rotateY(ang)	Gira em torno do eixo Y (de lado)
rotateZ(ang)	Igual a <code>rotate()</code> — gira na "tela"
rotate3d(x, y, z, ang)	Gira em torno de um vetor arbitrário
perspective(n)	Define perspectiva (use no PAI ou aqui mesmo)

10.4 Propriedades auxiliares (essenciais)

```
.elem {
  transform: rotate(45deg);

  transform-origin: top left;    /* ponto de origem
    valores: center | top | bottom | left | right | x y | x y z
    padrão: 50% 50% (centro) */

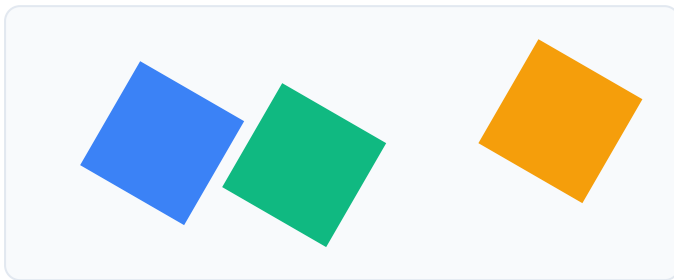
  transform-style: preserve-3d;  /* filhos respeitam o 3D
    (sem isto, filhos viram "achataados") */

  perspective: 1000px;          /* profundidade da cena
    quanto MENOR, mais exagerado o efeito 3D */

  perspective-origin: center top; /* de onde o observador olha */

  backface-visibility: hidden;   /* esconde o verso ao girar 180° */
}
```

10.5 transform-origin – mudando o ponto de pivô



```
/* Mesmo rotate(30deg), origens diferentes
*/
.a { transform-origin: center; }    /* padrão */
.b { transform-origin: top left; }
.c { transform-origin: bottom right; }
```

10.6 Receitas práticas de transform

a) Centralização absoluta (a melhor!)

```
.center {
  position: absolute;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}
```

b) Card que vira (flip card)

```
.flip-card {
  perspective: 1000px;
  width: 200px; height: 280px;
}
.flip-inner {
  position: relative;
  width: 100%; height: 100%;
  transition: transform 0.7s;
  transform-style: preserve-3d;
}
.flip-card:hover .flip-inner {
  transform: rotateY(180deg);
}
.front, .back {
  position: absolute; inset: 0;
  backface-visibility: hidden;
  border-radius: 12px;
}
.back { transform: rotateY(180deg); }
```

c) Botão com hover elegante

```
.btn {
  transition: transform .2s, box-shadow .2s;
}
.btn:hover {
  transform: translateY(-3px) scale(1.02);
  box-shadow: 0 10px 20px rgba(0,0,0,.2);
}
.btn:active {
  transform: translateY(0) scale(0.98);
}
```

d) Card 3D que segue o mouse (estilo)

```
.card {
  transform: perspective(800px) rotateX(10deg) rotateY(-15deg);
  transform-style: preserve-3d;
  transition: transform .3s;
}
.card:hover {
  transform: perspective(800px) rotateX(0) rotateY(0);
}
```

e) Tooltip com seta

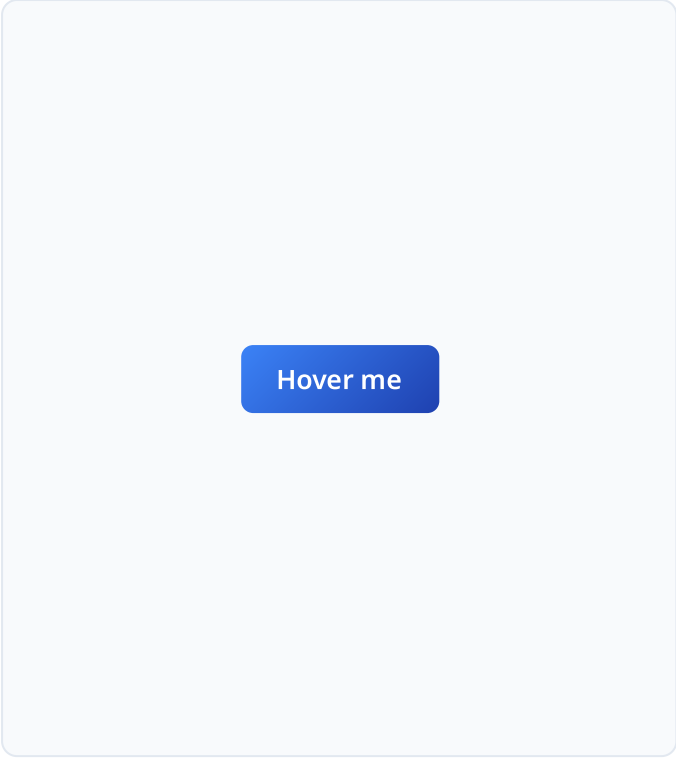
```
.tooltip {  
  position: absolute;  
  bottom: calc(100% + 8px);  
  left: 50%;  
  transform: translateX(-50%); /* centraliza */  
}  
.tooltip::after {  
  content: "";  
  position: absolute; top: 100%; left: 50%;  
  transform: translateX(-50%) rotate(45deg); /* seta */  
  width: 10px; height: 10px;  
  background: inherit;  
}
```

f) Ícone que pulsa

```
@keyframes pulse {  
  0%, 100% { transform: scale(1); }  
  50%    { transform: scale(1.15); }  
}  
.heart { animation: pulse 1.2s ease-in-out infinite; }
```

11. Transitions – animação suave entre estados

Define uma **animação automática** quando uma propriedade muda de valor (ex.: `:hover`, classe via JS).



Hover me

```
.btn {
  background: #3b82f6;
  transform: translateY(0);

  transition-property: background, transform, box-shadow;
  transition-duration: 0.3s;
  transition-timing-function: ease-in-out;
  transition-delay: 0s;

  /* SHORTHAND: prop duration timing delay */
  transition: all 0.3s ease-in-out;
}

.btn:hover {
  background: #1e40af;
  transform: translateY(-3px);
  box-shadow: 0 10px 20px rgba(0,0,0,.2);
}
```

11.1 Animar múltiplas propriedades com tempos diferentes

```
.card {
  transition:
    background 0.4s ease,
    transform 0.2s ease-out,
    box-shadow 0.6s ease-in;
}
```

11.2 Timing functions (curvas de aceleração)

- **linear** — velocidade constante (mecânico, robótico)
- **ease** — padrão. Lento, rápido, lento (natural)
- **ease-in** — começa devagar e acelera (sai da tela)
- **ease-out** — começa rápido e desacelera (entra na tela) — o melhor para UI
- **ease-in-out** — combina os dois
- **cubic-bezier(x1,y1,x2,y2)** — curva customizada (use cubic-bezier.com)
- **steps(n)** — animação em passos discretos (sprite sheets, máquina de escrever)

```
/* Bounce divertido */
transition: transform 0.4s cubic-bezier(0.68, -0.55, 0.265, 1.55);

/* Efeito de máquina de escrever */
transition: width 2s steps(20);
```

Atenção: nem todas as propriedades são animáveis. Você só pode interpolar valores numéricos (px, cores, opacity, transform). `display: none → block` não anima — use `opacity` + `visibility` + `transform`.

12. Animations e @keyframes

Diferente das transitions, `animation` roda **automaticamente** (sem precisar de hover) e permite vários quadros-chave.



0%



33%



66%



100%

```
@keyframes slide-in {
  0% { opacity: 0; transform: translateX(-30px)
scale(.5); }
  50% { opacity: 0.6; }
  100% { opacity: 1; transform: translateX(0)
scale(1); }
}

/* OU com from/to (só 0% e 100%) */
@keyframes fade {
  from { opacity: 0; }
  to { opacity: 1; }
}

.box {
  animation-name: slide-in;
  animation-duration: 1s;
  animation-timing-function: ease-out;
  animation-delay: 0.2s;
  animation-iteration-count: infinite; /* ou número
*/
  animation-direction: alternate; /* normal |
reverse | alternate */
  animation-fill-mode: forwards; /* mantém o
estado final */
  animation-play-state: running; /* running |
paused */

  /* SHORTHAND */
  animation: slide-in 1s ease-out 0.2s infinite
alternate forwards;
}
```

12.1 fill-mode (importantíssimo)

Valor	Efeito
none (padrão)	Volta ao estado original ao terminar (estranho)
forwards	Mantém o estado do último quadro
backwards	Aplica o primeiro quadro durante o delay
both	Combina forwards + backwards

12.2 Biblioteca de animações prontas

Fade in

```
@keyframes fadeIn {  
  from { opacity: 0; transform: translateY(10px); }  
  to   { opacity: 1; transform: translateY(0); }  
}  
.item { animation: fadeIn 0.5s ease-out both; }
```

Spinner (loading)

```
@keyframes spin {  
  to { transform: rotate(360deg); }  
}  
.loader {  
  width: 40px; height: 40px;  
  border: 4px solid #e2e8f0;  
  border-top-color: #3b82f6;  
  border-radius: 50%;  
  animation: spin 0.8s linear infinite;  
}
```

Pulse (chamando atenção)

```
@keyframes pulse {  
  0%, 100% { transform: scale(1); opacity: 1; }  
  50%      { transform: scale(1.05); opacity: 0.85; }  
}  
.live { animation: pulse 1.5s ease-in-out infinite; }
```

Shake (erro)

```
@keyframes shake {  
  0%, 100% { transform: translateX(0); }  
  25%    { transform: translateX(-6px); }  
  75%    { transform: translateX(6px); }  
}  
.error { animation: shake 0.3s ease-in-out 3; }
```

Bounce

```
@keyframes bounce {  
  0%, 20%, 50%, 80%, 100% { transform: translateY(0); }  
  40% { transform: translateY(-20px); }  
  60% { transform: translateY(-10px); }  
}
```

Typewriter (máquina de escrever)

```
@keyframes type {  
  from { width: 0; }  
  to   { width: 100%; }  
}  
.typing {  
  overflow: hidden;  
  white-space: nowrap;  
  border-right: 2px solid;  
  animation: type 3s steps(30) forwards;  
}
```

12.3 Pausar com hover

```
.carousel { animation: scroll 20s linear infinite; }  
.carousel:hover { animation-play-state: paused; }
```

13. Filters – efeitos visuais

Aplica efeitos gráficos (blur, brilho, saturação...) ao elemento e seus filhos. Funciona em qualquer elemento, não só imagens.

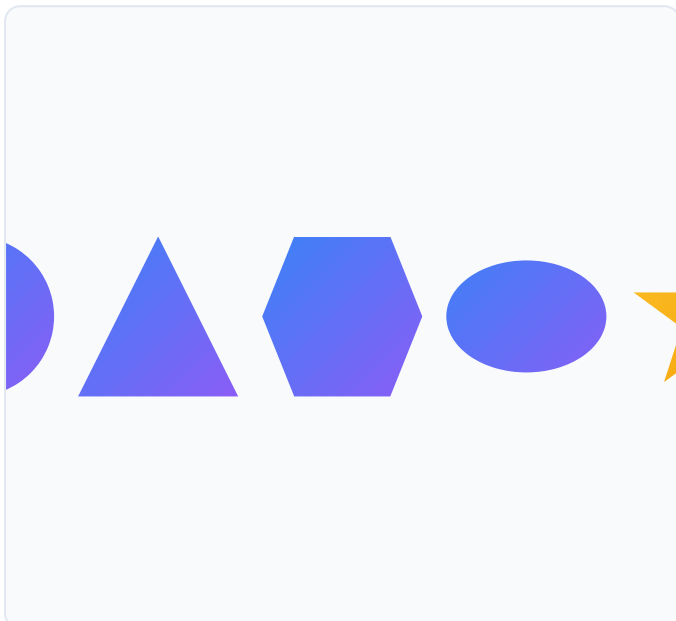


```
filter: blur(5px);
filter: brightness(1.5);           /* 1 = normal, >1 mais claro */
filter: contrast(150%);
filter: grayscale(100%);          /* 0% colorido, 100% PB */
filter: hue-rotate(90deg);        /* rotaciona matiz */
filter: invert(100%);             /* negativo */
filter: opacity(50%);
filter: saturate(200%);
filter: sepia(100%);              /* tom envelhecido */
filter: drop-shadow(2px 4px 6px black); /* respeita transparência */

/* COMBINAR múltiplos (espaço, não vírgula) */
filter: blur(2px) brightness(1.2) contrast(1.1);
```

drop-shadow vs box-shadow: `box-shadow` respeita o retângulo do elemento. `drop-shadow` respeita o formato real (transparência, PNG, SVG). Use `drop-shadow` para sombrear ícones.

14. clip-path – recortando elementos



```
clip-path: circle(50% at 50% 50%);
clip-path: ellipse(50% 35% at 50% 50%);
clip-path: polygon(50% 0, 100% 100%, 0 100%);
/* triângulo */

/* Hexágono */
clip-path: polygon(20% 0, 80% 0, 100% 50%,
                  80% 100%, 20% 100%, 0 50%);

/* Estrela 5 pontas */
clip-path: polygon(50% 0, 61% 35%, 98% 35%,
                  68% 57%,
                  79% 91%, 50% 70%, 21% 91%, 32%
                  57%,
                  2% 35%, 39% 35%);
```

15. Responsividade

15.1 Meta tag essencial

```
<meta name="viewport"
  content="width=device-width, initial-scale=1">
```

15.2 Media queries

```
/* MOBILE-FIRST (recomendado): base + min-width */
.container { padding: 12px; }

@media (min-width: 768px) {
  .container { padding: 24px; }
}
@media (min-width: 1200px) {
  .container { padding: 48px; max-width: 1140px; margin: 0 auto; }
}

/* Combinando condições */
@media (min-width: 768px) and (max-width: 1199px) { ... }
@media (min-width: 768px) and (orientation: landscape) { ... }

/* Preferências do usuário */
@media (prefers-color-scheme: dark) {
  body { background: #0f172a; color: #f1f5f9; }
}
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation: none !important;
    transition: none !important;
  }
}
```

15.3 Breakpoints comuns

Dispositivo	Breakpoint sugerido
Mobile pequeno	0 — 480px
Mobile / Tablet pequeno	481 — 767px
Tablet	768 — 1023px
Desktop	1024 — 1439px
Desktop grande	≥ 1440px

15.4 Container queries (moderno)

Responsivo baseado no **container**, não na viewport. Ideal para componentes reutilizáveis.

```
.card-wrapper {  
  container-type: inline-size;  
  container-name: card;  
}  
  
@container card (min-width: 400px) {  
  .card { display: flex; } /* só se o WRAPPER tiver ≥400px */  
}
```

16. Variáveis CSS (Custom Properties)

Permitem definir valores reutilizáveis com `--nome` e acessá-los com `var()`. Funcionam em tempo real (atualizáveis via JS) e respeitam cascata.

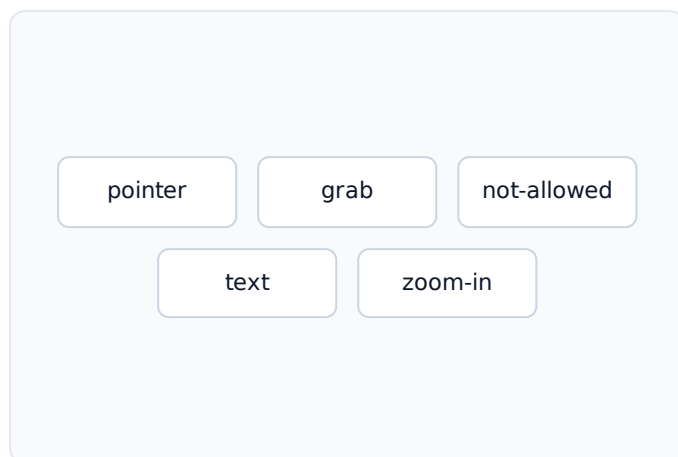
```
:root { /* escopo global */  
  --cor-primaria: #3b82f6;  
  --cor-texto: #0f172a;  
  --radius: 8px;  
  --espaco: 16px;  
  --shadow-sm: 0 1px 3px rgba(0,0,0,.1);  
}  
  
.btn {  
  background: var(--cor-primaria);  
  color: var(--cor-texto);  
  padding: var(--espaco);  
  border-radius: var(--radius);  
  box-shadow: var(--shadow-sm);  
}  
  
/* Fallback caso a variável não exista */  
.x { color: var(--cor-tema, #000); }  
  
/* Trocar tema escuro */  
.dark {  
  --cor-texto: #fff;  
  --cor-primaria: #60a5fa;  
}  
  
/* Calc com variáveis */  
.card { padding: calc(var(--espaco) * 2); }
```

17. Outras propriedades essenciais

17.1 overflow

```
overflow: visible; /* padrão, conteúdo "vaza" */
overflow: hidden; /* corta o que sobra */
overflow: scroll; /* sempre mostra barras */
overflow: auto; /* só mostra se precisar */
overflow-x: auto; overflow-y: hidden;
```

17.2 cursor



```
cursor: pointer; /* mão (link/botão) */
cursor: grab; /* arrastar */
cursor: grabbing;
cursor: not-allowed;
cursor: text;
cursor: zoom-in / zoom-out;
cursor: wait; /* ampulheta */
cursor: crosshair;
cursor: url("cursor.png"), auto; /* customizado */
```

17.3 object-fit e object-position

Controla como uma imagem/vídeo preenche seu container.

```
img {
  width: 200px; height: 200px;
  object-fit: cover; /* preenche, corta o que sobra */
  object-fit: contain; /* cabe inteiro (pode sobrar fundo) */
  object-fit: fill; /* estica (deforma) */
  object-fit: none; /* tamanho real */
  object-fit: scale-down;
  object-position: center; /* foco do recorte */
}
```

17.4 aspect-ratio

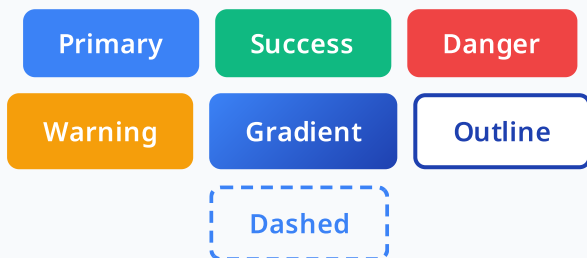
```
.video {
  width: 100%;
  aspect-ratio: 16 / 9; /* mantém proporção, calcula altura */
}
.avatar { aspect-ratio: 1; width: 80px; } /* quadrado */
```

17.5 Outras úteis

Propriedade	Valores comuns	Descrição
visibility	visible hidden collapse	hidden mantém o espaço (display:none não)
opacity	0 a 1	Transparência (mantém eventos)
pointer-events	auto none	none ignora cliques/hover
user-select	none text all auto	Permitir ou não selecionar texto
resize	none both horizontal vertical	Redimensionar com o mouse
scroll-behavior	auto smooth	Rolagem suave em links #
mix-blend-mode	multiply screen overlay darken	Fusão com o fundo
will-change	transform opacity	Otimização — usar com moderação
caret-color	cor	Cor do cursor em inputs
accent-color	cor	Cor de checkbox, radio, range

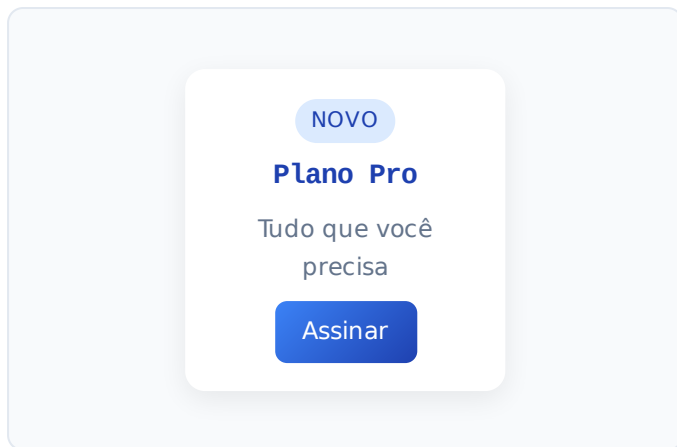
18. Componentes prontos para copiar

18.1 Botões variados



```
.btn {  
  padding: 10px 20px;  
  border: 0; border-radius: 6px;  
  font-weight: 600; cursor: pointer;  
  transition: all .2s;  
}  
.btn-primary { background: #3b82f6; color: #fff;  
}  
.btn-success { background: #10b981; color: #fff;  
}  
.btn-danger { background: #ef4444; color: #fff;  
}  
.btn-gradient { background: linear-  
  gradient(135deg, #3b82f6, #1e40af); color: #fff;  
}  
.btn-outline { background: #fff; color: #1e40af;  
  border: 2px solid #1e40af; }  
.btn:hover { transform: translateY(-2px); box-  
  shadow: 0 6px 14px rgba(0,0,0,.15); }
```

18.2 Card moderno (hover 3D)



```
.card {  
  padding: 20px; background: #fff;  
  border-radius: 12px;  
  box-shadow: 0 4px 14px rgba(0,0,0,.08);  
  transition: transform .3s, box-shadow .3s;  
}  
.card:hover {  
  transform: translateY(-6px);  
  box-shadow: 0 14px 30px rgba(0,0,0,.18);  
}
```

18.3 Formulário com foco animado

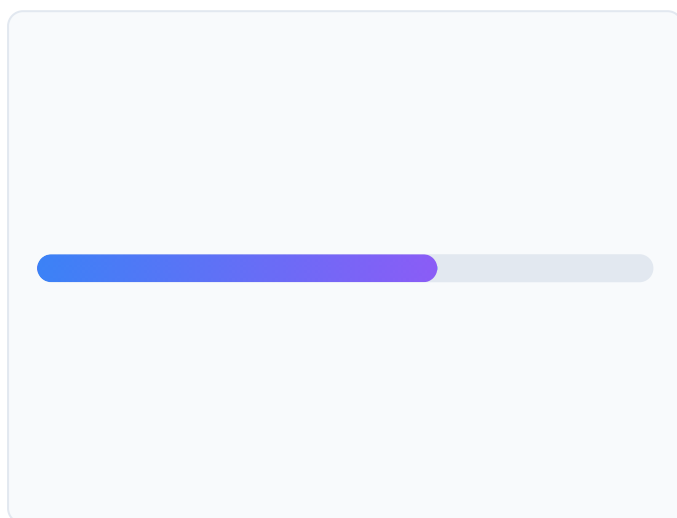
Email

Senha

Entrar

```
input {  
  padding: 8px 10px;  
  border: 2px solid #e2e8f0;  
  border-radius: 6px;  
  transition: border-color .2s, box-shadow .2s;  
  outline: none;  
}  
input:focus {  
  border-color: #3b82f6;  
  box-shadow: 0 0 0 3px rgba(59,130,246,.2);  
}
```

18.4 Barra de progresso



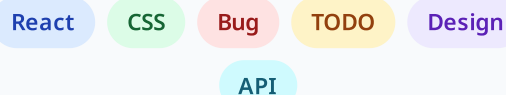
```
.progress {  
  width: 100%; height: 14px;  
  background: #e2e8f0;  
  border-radius: 999px;  
  overflow: hidden;  
}  
.progress .bar {  
  height: 100%; width: 65%;  
  background: linear-gradient(90deg, #3b82f6,  
    #8b5cf6);  
  transition: width .4s ease;  
}
```

18.5 Toggle switch (apenas com CSS)



```
.switch {  
  position: relative;  
  width: 50px; height: 26px;  
  background: #cbd5e1;  
  border-radius: 999px;  
  transition: background .25s;  
}  
.switch::after {  
  content: "";  
  position: absolute; top: 3px; left: 3px;  
  width: 20px; height: 20px;  
  background: #fff; border-radius: 50%;  
  transition: left .25s;  
}  
.switch.on { background: #3b82f6; }  
.switch.on::after { left: 27px; }
```

18.6 Tags / badges



```
.tag {  
  padding: 4px 10px;  
  border-radius: 999px;  
  font-size: 11px; font-weight: 600;  
}  
.tag-blue { background: #dbeafe; color: #1e40af; }  
.tag-green { background: #dcfce7; color: #166534; }  
.tag-red { background: #fee2e2; color: #991b1b; }
```

19. Boas práticas

- Use **classes** em vez de IDs para estilizar (reutilizáveis).
- Use `box-sizing: border-box` globalmente.
- Prefira **rem/em** em vez de **px** para tipografia (acessibilidade).
- Evite `!important` — sinal de CSS mal estruturado.
- Use **variáveis CSS** para cores e espaçamentos consistentes.
- **Mobile-first**: escreva o CSS base para mobile e amplie com `min-width`.
- Em animações use **transform** e **opacity** (GPU = 60fps).
- Use nomes de classes descritivos (BEM: `.card_title--large`).

- Mantenha especificidade baixa e plana.
- Respeite `prefers-reduced-motion`.
- Teste em **vários tamanhos** de tela (DevTools).

19.1 Reset CSS recomendado

```
*, *::before, *::after {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
html { scroll-behavior: smooth; -webkit-text-size-adjust: 100%; }  
body {  
  font-family: system-ui, -apple-system, sans-serif;  
  line-height: 1.6;  
  color: #0f172a;  
  background: #fff;  
}  
img, video { max-width: 100%; display: block; }  
a { color: inherit; text-decoration: none; }  
button { font: inherit; cursor: pointer; }  
input, textarea, select { font: inherit; }  
:focus-visible { outline: 2px solid #3b82f6; outline-offset: 2px; }
```

20. Mini-projeto: card de produto

completo

```
<article class="product">
  <div class="product-img"></div>
  <div class="product-info">
    <span class="tag">Novidade</span>
    <h3>Tênis Nova Era</h3>
    <p class="price">R$ 299,90</p>
    <button class="btn">Adicionar ao carrinho</button>
  </div>
</article>

<style>
.product {
  width: 280px;
  background: #fff;
  border-radius: 16px;
  overflow: hidden;
  box-shadow: 0 4px 14px rgba(0,0,0,.08);
  transition: transform .35s ease, box-shadow .35s ease;
}
.product:hover {
  transform: translateY(-8px) scale(1.01);
  box-shadow: 0 20px 40px rgba(0,0,0,.18);
}
.product-img { aspect-ratio: 4/3; overflow: hidden; }
.product-img img {
  width: 100%; height: 100%;
  object-fit: cover;
  transition: transform .5s ease;
}
.product:hover .product-img img { transform: scale(1.08); }

.product-info { padding: 18px; }
.tag {
  display: inline-block;
  padding: 3px 10px;
  background: #dbeafe;
  color: #1e40af;
  border-radius: 999px;
  font-size: 11px;
  font-weight: 600;
}
.product h3 { margin: 8px 0 4px; color: #0f172a; }
.price {
  font-size: 22px; font-weight: 700;
  background: linear-gradient(90deg, #3b82f6, #8b5cf6);
  -webkit-background-clip: text;
  color: transparent;
}
.btn {
  width: 100%; margin-top: 12px;
  padding: 12px;
  background: linear-gradient(135deg, #3b82f6, #1e40af);
}
```

```
color: #fff;  
border: 0; border-radius: 8px;  
font-weight: 600; cursor: pointer;  
transition: transform .2s, filter .2s;  
}  
.btn:hover { transform: translateY(-2px); filter: brightness(1.1); }  
.btn:active { transform: translateY(0); }  
</style>
```

Fim da apostila. Pratique cada exemplo no navegador – o melhor jeito de aprender CSS é experimentando!